Bài tập

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation, PillowWriter, FFMpegWriter

# --- Bandit setup ---
np.random.seed(42)
true_probs = [0.005, 0.01, 0.02]
n_arms = len(true_probs)

#Số vòng
n_rounds = 120

cost_per_pull = 1000.0

# --- Policies setup ---
epsilon = 0.1
policies = ['eps_greedy', 'ucb', 'thompson']
titles = ["ε-Greedy (ε=0.1)", "UCB1", "Thompson Sampling"]

# --- arm color ---
arm_colors = ['tab:red', 'tab:green', 'tab:orange', 'tab:purple']

# --- Initialize estimates ---
Q_eps = np.zeros(n_arms)
N_eps = np.zeros(n_arms)
Q_ucb = np.zeros(n_arms)
N_ucb = np.zeros(n_arms)
alpha_ts = np.ones(n_arms)
beta_ts = np.ones(n_arms)

# --- Histories ---
histories = {p: [] for p in policies}
money_history = {
    p: [] for p in policies
}

# --- Policy functions ---
def select_epsilon_greedy():
    if np.random.rand() < epsilon:
        return np.random.randint(n_arms)
    return np.argmax(Q_eps)

def select_ucb(t):
```

```
    if 0 in N_ucb:
        return np.argmin(N_ucb)
    ucb_values = Q_ucb + np.sqrt(2 * np.log(t + 1) / N_ucb)
    return np.argmax(ucb_values)

def select_thompson():
    samples = np.random.beta(alpha_ts, beta_ts)
    return np.argmax(samples)

def update_estimates(policy, a, r):
    if policy == 'eps_greedy':
        N_eps[a] += 1
        Q_eps[a] += (r - Q_eps[a]) / N_eps[a]
    elif policy == 'ucb':
        N_ucb[a] += 1
        Q_ucb[a] += (r - Q_ucb[a]) / N_ucb[a]
    elif policy == 'thompson':
        alpha_ts[a] += r
        beta_ts[a] += (1 - r)

def run_one_step(policy, t):
    if policy == 'eps_greedy':
        a = select_epsilon_greedy()
    elif policy == 'ucb':
        a = select_ucb(t)
    else:
        a = select_thompson()

    # Xác định có trúng hay không
    win = np.random.rand() < true_probs[a]

    # Reward dùng cho thuật toán Bandit
    r = int(win)

    # Tiền thực tế dùng để tính lợi nhuận
    if win:
        prize = np.random.uniform(5_000_000, 10_000_000)
        money = prize - cost_per_pull
    else:
        money = -cost_per_pull

    update_estimates(policy, a, r)
    histories[policy].append((t, a, r, money))
    if len(money_history[policy]) == 0:
        money_history[policy].append(money)
```

```
    else:
        money_history[policy].append(
            money_history[policy][-1] + money
        )

# --- Figure setup ---
fig, axes = plt.subplots(3, 2, figsize=(10, 8))
plt.tight_layout(pad=3.0)

bars = []

for i, policy in enumerate(policies):
    ax_top, ax_bottom = axes[i]

    # --- scatter actions ---
    ax_top.set_xlim(0, n_rounds)
    ax_top.set_ylim(-0.5, n_arms - 0.5)
    ax_top.set_yticks(range(n_arms))
    ax_top.set_yticklabels([f"Arm {j} (θ={true_probs[j]:.3f})" for j in
range(n_arms)])
    ax_top.set_title(titles[i])

    # --- estimated means ---
    ax_bottom.set_ylim(0, 1)
    bars_plot = ax_bottom.bar(range(n_arms), [0]*n_arms, color=arm_colors,
alpha=0.7)
    bars.append(bars_plot)
    ax_bottom.set_xticks(range(n_arms))
    ax_bottom.set_ylabel("Estimated θ")

# --- Animation update ---
def update(frame):
    for i, policy in enumerate(policies):
        run_one_step(policy, frame)
        t_list = [h[0] for h in histories[policy]]
        arm_list = [h[1] for h in histories[policy]]
        reward_list = [h[2] for h in histories[policy]]

        for coll in list(axes[i, 0].collections):
            coll.remove()

        # draw history
        for arm in range(n_arms):
            arm_times = [t for t, a in zip(t_list, arm_list) if a == arm]
            arm_rewards = [r for a, r in zip(arm_list, reward_list) if a == arm]
```

```
            x_empty = [t for t, r in zip(arm_times, arm_rewards) if r == 0]
            y_empty = [arm] * len(x_empty)
            x_filled = [t for t, r in zip(arm_times, arm_rewards) if r == 1]
            y_filled = [arm] * len(x_filled)

            axes[i, 0].scatter(x_empty, y_empty, facecolors='none',
                               edgecolors=arm_colors[arm], s=50, linewidths=1.5)
            axes[i, 0].scatter(x_filled, y_filled, color=arm_colors[arm], s=50)

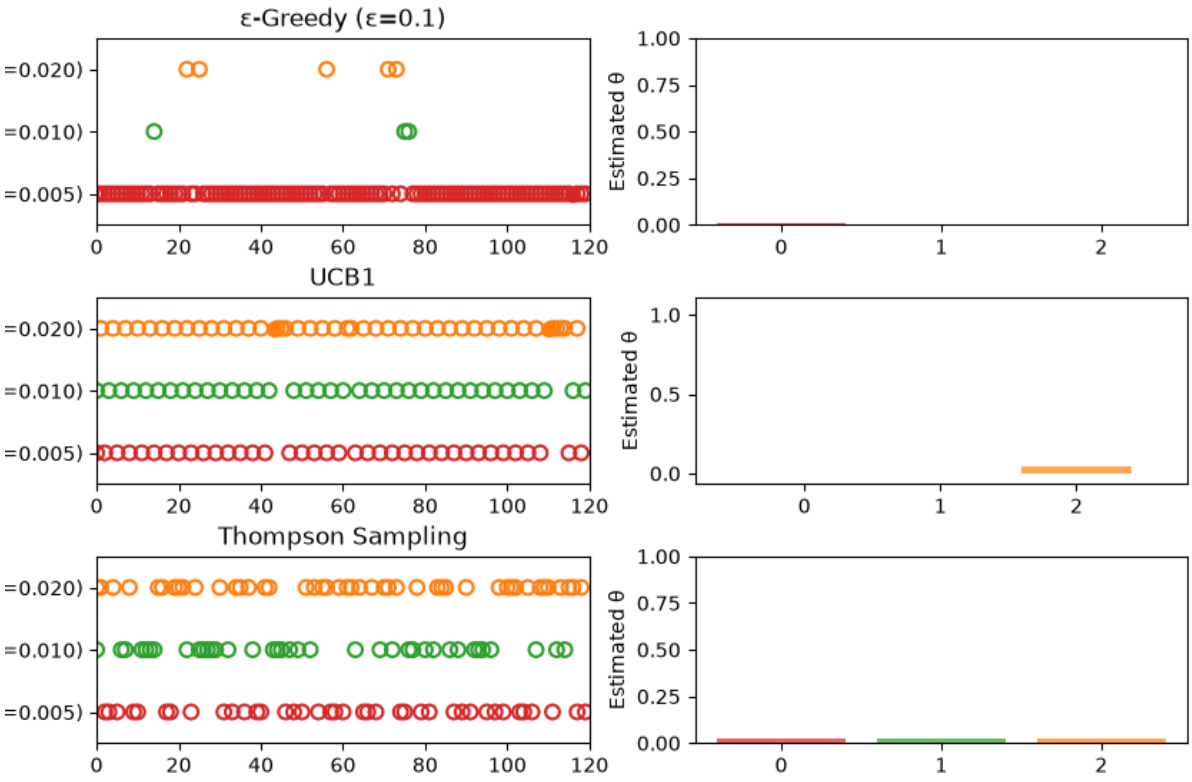
        # update estimate
        if policy == 'eps_greedy':
            est = Q_eps.copy()
        elif policy == 'ucb':
            est = Q_ucb.copy()
        else:
            est = alpha_ts / (alpha_ts + beta_ts)

        for j, b in enumerate(bars[i]):
            b.set_height(est[j])
    return [b for bar in bars for b in bar]

# --- Run animation ---
anim = FuncAnimation(fig, update, frames=n_rounds, interval=120, blit=False,
repeat=False)

# --- Save GIF & MP4 ---
# anim.save("bandit_compare.gif", writer=PillowWriter(fps=15))
# anim.save("bandit_compare.mp4", writer=FFMpegWriter(fps=15))

plt.show()
```



Bài tập nâng cao

```python
import numpy as np

# CÂU A

names = ["Banner", "Footer", "Side page", "Pop-up", "Unskippable"]

ctr = [0.05, 0.08, 0.12, 0.03, 0.10]

# Chi phí mỗi lần hiển thị, đơn vị cent
cost = [1.0, 1.2, 1.5, 0.8, 1.3]

budget = 2000 * 100

efficiency = []

for i in range(len(names)):
    efficiency.append(ctr[i] / cost[i])

best = efficiency.index(max(efficiency))
print("===== CÂU A =====")
```

```python
print("Efficiency của từng loại:")

for i in range(len(names)):
    print(names[i], ":", efficiency[i])

print("\nLoại quảng cáo hiệu quả nhất:", names[best])

# CÂU B

print("\n===== CÂU B =====")
np.random.seed(42)
impressions = 1000
clicks = []

for i in range(len(names)):
    # Số click thực tế
    click = np.random.binomial(impressions, ctr[i])
    clicks.append(click)

# Tính CTR thực tế
actual_ctr = []

for i in range(len(names)):
    actual_ctr.append(clicks[i] / impressions)


actual_efficiency = []

for i in range(len(names)):
    actual_efficiency.append(actual_ctr[i] / cost[i])

print("Kết quả thực tế:")

for i in range(len(names)):
    print(
        names[i],
        "| Clicks:", clicks[i],
        "| CTR:", actual_ctr[i],
        "| Efficiency:", actual_efficiency[i])

best_actual = actual_efficiency.index(max(actual_efficiency))

print("\nLoại có Efficiency thực tế cao nhất:",
      names[best_actual])
```